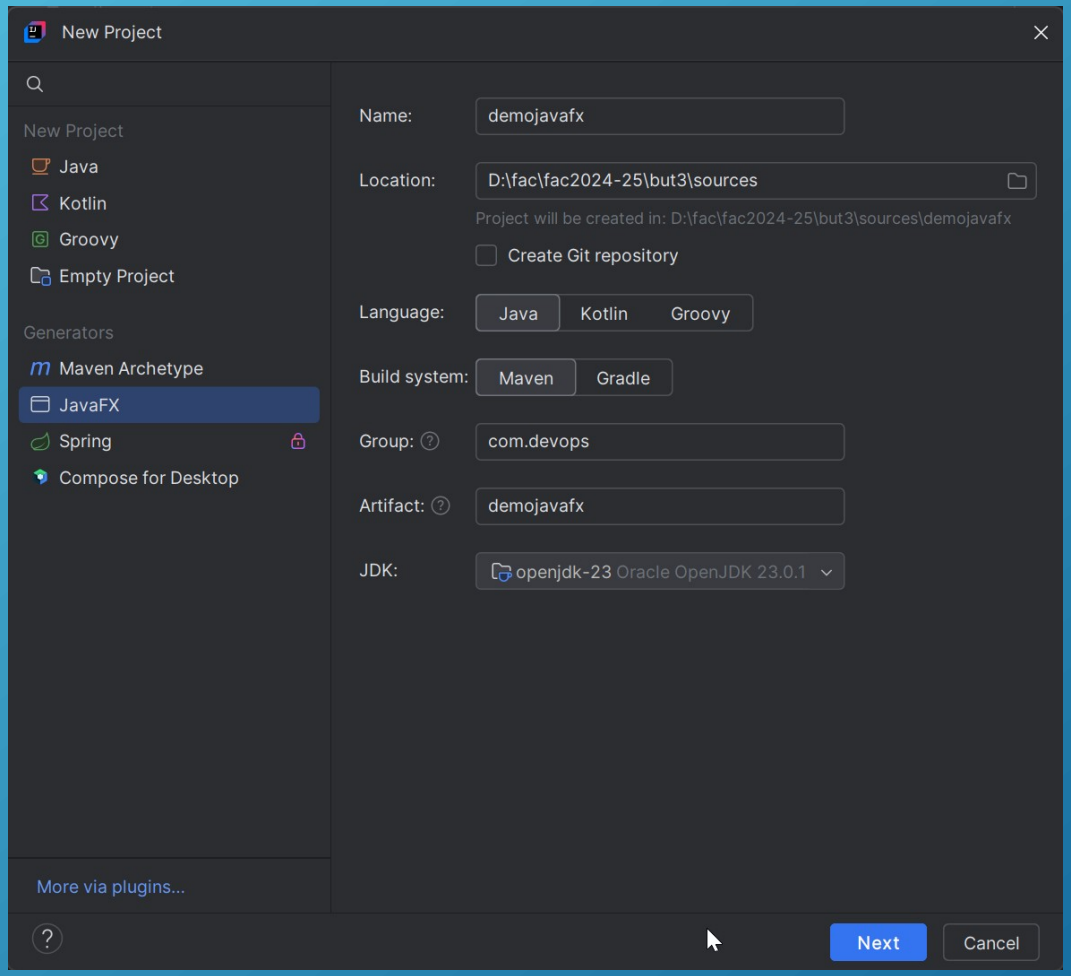
LICENCE PRO DEVOPS

Java FX

IUT Informatique – Université Claude Bernard Lyon 1

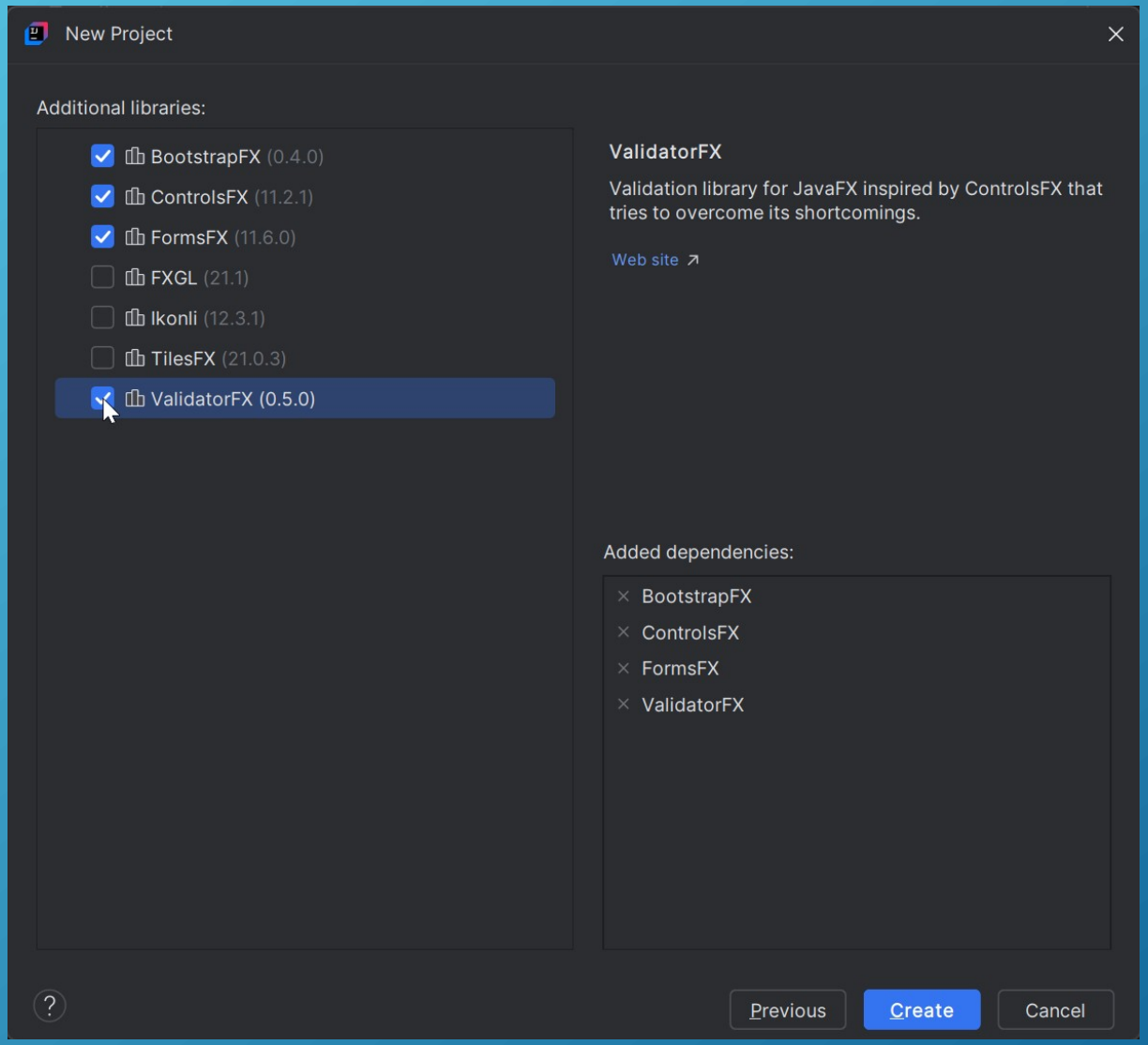
JAVAFX - INSTALLATION

Utilisation de intellij - Maven

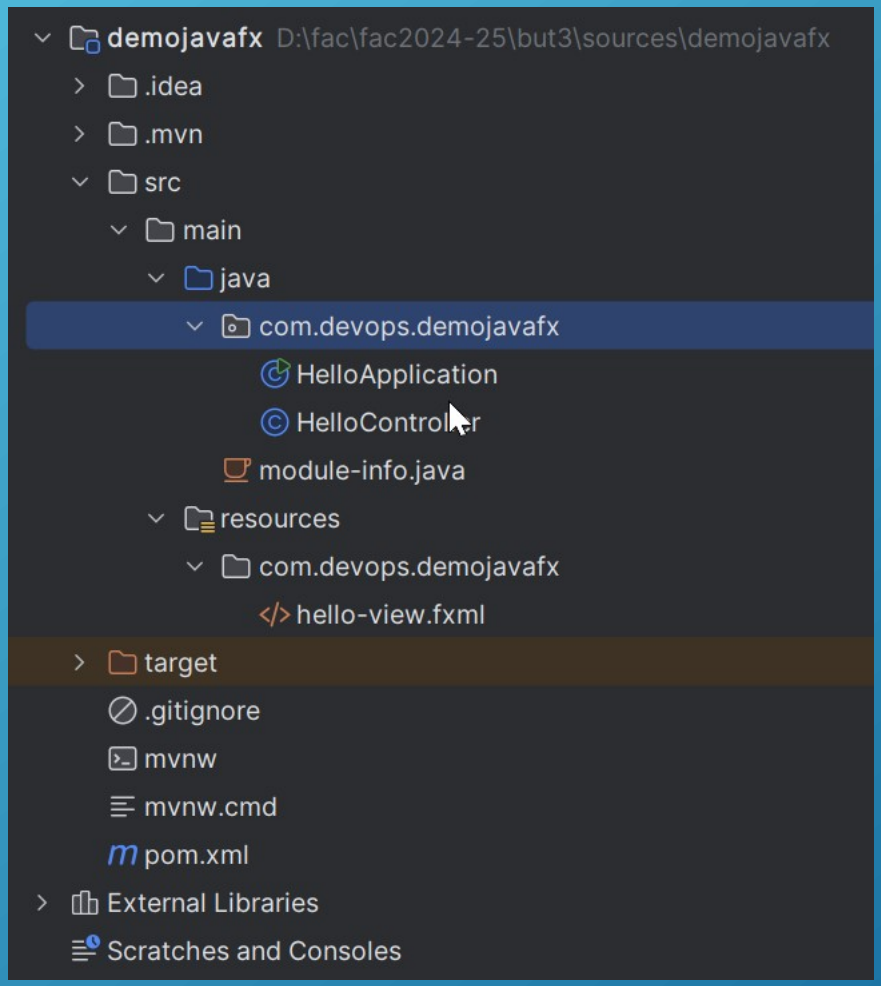


JAVAFX - INSTALLATION

- Les librairies seront intégrées dans le fichier pom.xml.

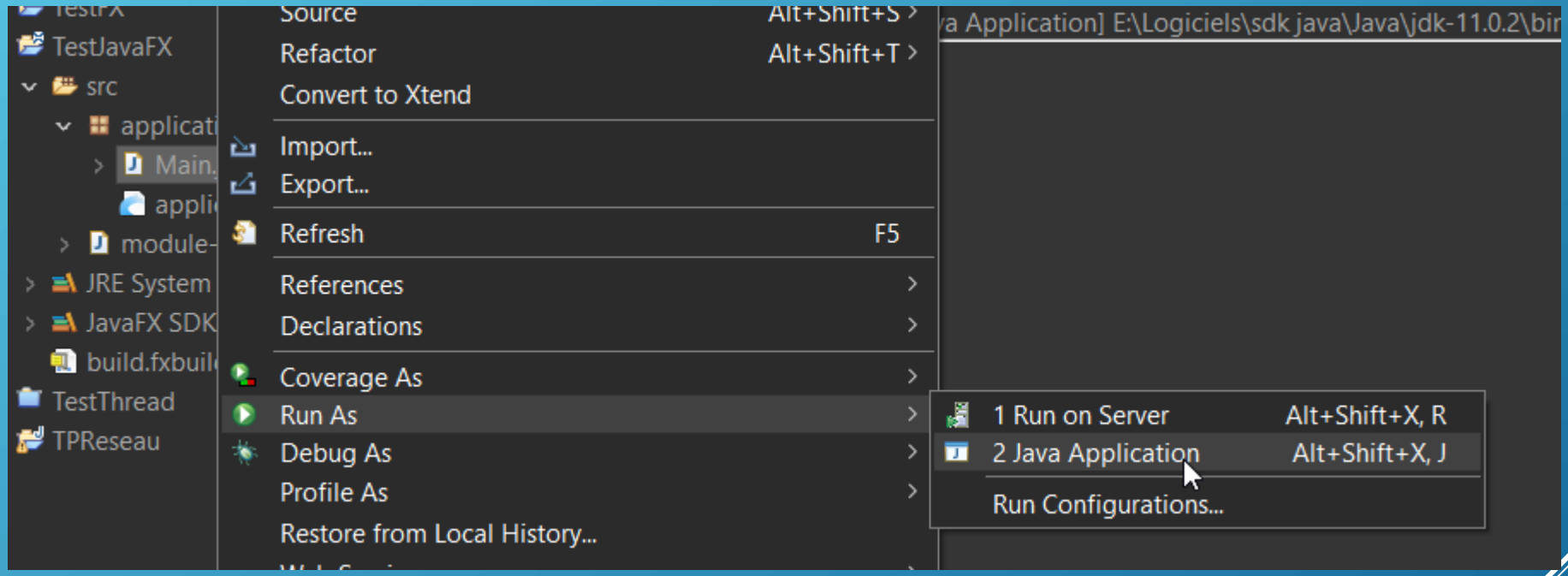


JAVAFX - INSTALLATION

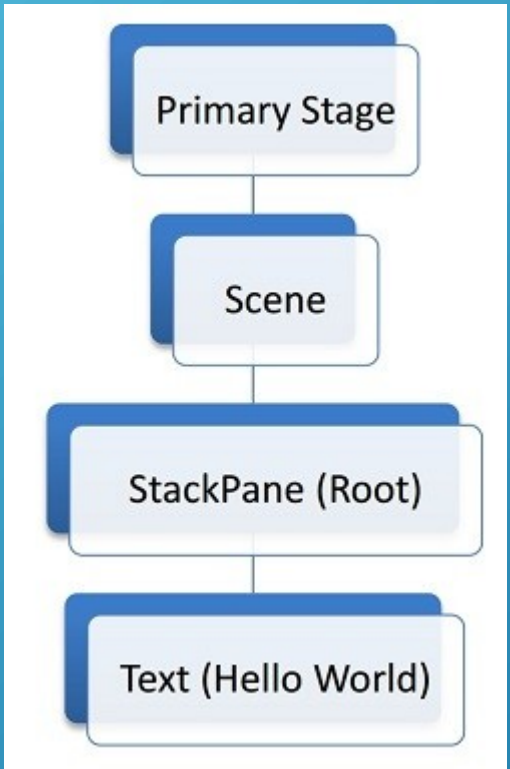


JAVAFX

Lancer une application



JAVAFX



- **aide**
  - https://www.javatpoint.com/javafx-tutorial
  - https://www.tutorialkart.com/javafx/javafx-textfield/
-

JAVAFX

Le fichier fxml

```
public class HelloApplication extends Application {
    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader(HelloApplication.class.getResource( name: "hello-view.fxml"));
        Scene scene = new Scene(fxmlLoader.load(), v: 320, v1: 240);
        stage.setTitle("Hello!");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) { launch(); }
```

JAVAFX

Le fichier fxml

https://examples.javacodegeeks.com/desktop-java/javafx/fxml/javafx-fxml-controller-example/

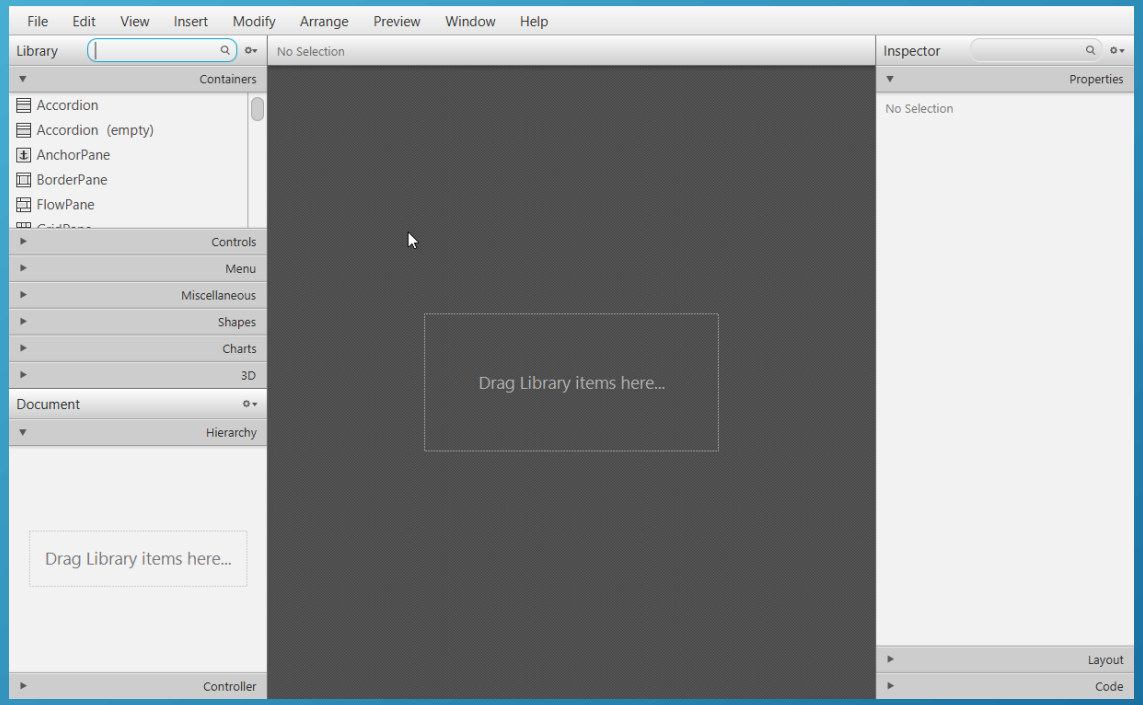
```
<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.VBox?>

<?import javafx.scene.control.Button?>
<VBox alignment="CENTER" spacing="20.0" xmlns:fx="http://javafx.com/fxml"
      fx:controller="com.devops.demojavafx3.HelloController">
    <padding>
        <Insets bottom="20.0" left="20.0" right="20.0" top="20.0"/>
    </padding>

    <Label fx:id="welcomeText"/>
    <Button text="Hello!" onAction="#onHelloButtonClick"/>
</VBox>
```

JAVAFX

- **Installation de sceneBuilder dans intellj:**
  - https://www.jetbrains.com/help/idea/opening-fxml-files-in-javafx-scene-builder.html
- **Lien sceneBuilder** https://gluonhq.com/products/scene-builder/#download



JAVAFX

Bonnes pratiques

- Séparer la logique métier et l'interface utilisateur : traitement de données, logique d'application.
- Créer des méthodes pour configurer les composants

```
private Button createButton() {
    Button button = new Button("Cliquez-moi");
    button.setOnAction(e -> System.out.println("Bouton cliqué!"));
    return button;
}
```

- Utilisation des layout pour organiser les composants : classes de disposition (layouts) telles que VBox, HBox, GridPane, etc., pour organiser vos composants de manière structurée.

```
VBox vbox = new VBox();
vbox.setSpacing(10); // Espace entre les composants
vbox.getChildren().addAll(button1, button2, textField);
```

- Garder le style séparé ex : scene.getStylesheets().add("style.css");

JAVAFX

Le contrôleur

```java
public void start(Stage primaryStage) {
    // Création du layout principal (exemple avec BorderPa
    BorderPane root = new BorderPane();

    // Instancier le contrôleur et configurer les composar
    MyController controller = new MyController();
    root.setCenter(controller.getView()); // Ajout de la v

    // Création et configuration de la scène
    Scene scene = new Scene(root, v: 600, v1: 400);
    primaryStage.setTitle("Application JavaFX avec Contrôl
    primaryStage.setScene(scene);
    primaryStage.show();
}
```

```java
public class MyController {
    // Vue (layout qui contiendra les éléments du contrôleur)
    private VBox view;  3 usages
    private Label label;  3 usages
    private Button button;  3 usages

    public MyController() {
        // Initialisation des composants de l'interface
        label = new Label( s: "Bonjour, JavaFX!");
        button = new Button( s: "Cliquez-moi");

        // Ajout d'un gestionnaire d'événements au bouton
        button.setOnAction(e -> onButtonClick());

        // Création et configuration du layout pour la vue
        view = new VBox( v: 10);
        view.getChildren().addAll(label, button);
    }

    // Méthode pour gérer le clic sur le bouton
    private void onButtonClick() { label.setText("Le bouton a été cliqué !"); }

    // Méthode pour obtenir la vue du contrôleur
    public VBox getView() {  1 usage
        return view;
    }
}
```